

Deep Learning Laboratory

Assignment #7

Siddharth Narayan Mishra
123CS0189

September 16, 2026

Contents

1	Objective	2
2	Theory	2
2.1	The three aggregators	2
2.2	Breakdown point and influence	3
2.3	Gradients of a selection	3
2.4	Effective sample usage	3
3	Experimental Setup	4
4	Algorithm	4
5	Implementation	5
6	Results	6
7	Analysis	10
7.1	Clean data costs the robust aggregators accuracy	10
7.2	Trimming pays for itself only when the tail is corrupted	10
7.3	Outlier sensitivity follows the breakdown point exactly	10
7.4	The median does produce gradients, but from one sample	11
7.5	Batch size separates the aggregators by an order of magnitude	11
8	Research Hypothesis	11
9	Conclusion	12

Robust Aggregation of Per Sample Squared Errors

1 Objective

- Train a fixed convolutional network on MNIST with three different aggregations of the per sample squared error, namely the mean, the median and the ten percent trimmed mean, under one identical optimization protocol.
- Measure how the choice of aggregator changes convergence speed, final clean accuracy, and the degradation caused by label noise and by input noise.
- Quantify the sensitivity of each aggregator to a small number of grossly inflated squared errors inside a mini batch.
- Determine how many samples of a mini batch actually contribute to the parameter gradient under each aggregator, and in particular whether `backward()` produces a usable gradient for the median.
- Establish how the three aggregators respond to a change of batch size, and propose a research hypothesis from the observed behaviour.

2 Theory

2.1 The three aggregators

Let a mini batch contain n samples. For sample i let $y_i \in \{0, 1\}^{10}$ be the one hot target and $\hat{y}_i = \sigma(z_i) \in (0, 1)^{10}$ the sigmoid output of the network. The per sample squared error is the squared Euclidean distance between the two vectors,

$$s_i = \sum_{c=1}^{10} (y_{ic} - \hat{y}_{ic})^2. \quad (1)$$

The three losses differ only in how the vector $s = (s_1, \dots, s_n)$ is reduced to a scalar:

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n s_i, \quad (2)$$

$$L_{\text{MedSE}} = \text{median}(s_1, \dots, s_n), \quad (3)$$

$$L_{\text{T-MSE}} = \frac{1}{n-k} \sum_{j=1}^{n-k} s_{(j)}, \quad k = \lfloor 0.1n \rfloor, \quad (4)$$

where $s_{(1)} \leq \dots \leq s_{(n)}$ is the sorted batch and the k largest entries are discarded. For $n = 64$ this trims $k = 6$ samples and keeps 58.

2.2 Breakdown point and influence

Each aggregator is a statistic of the empirical distribution of s , so it inherits a breakdown point, that is the smallest fraction of arbitrarily corrupted entries that can drive the statistic to an arbitrary value. The mean has breakdown point 0, since a single entry sent to infinity sends the mean to infinity. The median has breakdown point $1/2$. The one sided 10% trimmed mean has breakdown point 0.1, because once more than k entries are corrupted at least one of them survives the trim.

This is the exact prediction tested in Experiment 4. Inflating m of the n squared errors by a factor α changes the mean by

$$\frac{\Delta L_{\text{MSE}}}{L_{\text{MSE}}} = (\alpha - 1) \frac{\sum_{i \in \mathcal{O}} s_i}{\sum_{i=1}^n s_i}, \quad (5)$$

which for $\alpha = 100$, $m = 2$ and roughly uniform errors is about $99 \times 2/64 \approx 309\%$. The median and the trimmed mean are affected only through a change of rank, not through the magnitude of the corruption.

2.3 Gradients of a selection

The gradient with respect to the parameters is obtained by the chain rule through the weights $w_i = \partial L / \partial s_i$:

$$\nabla_{\theta} L = \sum_{i=1}^n w_i \nabla_{\theta} s_i. \quad (6)$$

For the mean, $w_i = 1/n$ for every i , so all n samples contribute. For the trimmed mean, $w_i = 1/(n - k)$ on the kept samples and $w_i = 0$ on the trimmed ones. The median is a selection operator: with n even, `torch.median` returns the lower of the two central order statistics, which is a single element of the batch. It is therefore differentiable almost everywhere, with

$$w_i = \mathbb{I}[i = i_{\text{med}}], \quad (7)$$

so `backward()` does produce a gradient, but that gradient is exactly the gradient of one sample and carries no information about the other $n - 1$.

This has a direct consequence for gradient magnitude. If the per sample gradients $\nabla_{\theta} s_i$ are roughly uncorrelated with comparable norm g , then averaging gives $\|\nabla_{\theta} L_{\text{MSE}}\|_2 \approx g/\sqrt{n}$, whereas the median gives $\|\nabla_{\theta} L_{\text{MedSE}}\|_2 \approx g$. The median therefore produces larger and far noisier steps, and its step norm does not shrink as the batch grows.

2.4 Effective sample usage

A further consequence is that the median consumes exactly one sample per optimizer step regardless of n . Over E epochs on N training samples with batch size n there are EN/n steps, so the median run applies gradient information from only EN/n sample presentations while the mean uses all EN . The median is thus predicted to lose accuracy in proportion to the batch size, which is what Experiment 6 measures.

3 Experimental Setup

- Dataset: MNIST loaded with `torchvision.datasets.MNIST`. A fixed permutation with seed 42 selects 10000 training images and 2000 test images. The test split is never corrupted in any experiment.
- Preprocessing: images are kept in $[0, 1]$ while noise is injected, then normalized with mean 0.1307 and standard deviation 0.3081. Injecting the Gaussian noise before normalization keeps the prescribed standard deviation of 0.1 in the pixel domain where it is meaningful.
- Architecture: $\text{Conv}(1 \rightarrow 8, 3 \times 3) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \rightarrow \text{Conv}(8 \rightarrow 16, 3 \times 3) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \rightarrow \text{FC}(400 \rightarrow 10)$, followed by a sigmoid on the ten outputs. The spatial sizes are $28 \rightarrow 26 \rightarrow 13 \rightarrow 11 \rightarrow 5$, giving $16 \times 5 \times 5 = 400$ features into the fully connected layer.
- Optimization: plain SGD with learning rate 0.01, no momentum, no weight decay, batch size 64, 5 epochs, seed 42. The seed is reset immediately before every model is constructed and the data loader generator is seeded identically, so every run in the report starts from bit identical weights and sees the same batch order. Only Experiment 6 changes the batch size.
- Targets: one hot vectors of length 10 compared against the sigmoid outputs, with no softmax and no cross entropy anywhere.
- Label noise: a seeded subset of the training labels is replaced by $(y + u) \bmod 10$ with u drawn uniformly from $\{1, \dots, 9\}$, which guarantees the new label is wrong.
- Input noise: a seeded subset of the training images receives additive $\mathcal{N}(0, 0.1^2)$ noise per pixel. Labels are untouched and the values are not clipped.
- Reported test loss uses the same aggregator as training, evaluated per batch of 64 on the clean test set and averaged over batches.

4 Algorithm

1. Select the 10000 training and 2000 test samples with the seeded permutation and hold them as tensors in memory.
2. For each loss aggregator, reset the seed to 42, build the CNN, and train for 5 epochs with SGD. In every step compute s_i for the batch, reduce it with the chosen aggregator, call `backward()`, read $\|\nabla_{\theta} L\|_2$ over all parameters before the update, and then step.
3. At the end of each epoch evaluate the loss and the accuracy on the clean test set and record the epoch wise history.
4. Repeat the three runs with 10% and 20% of the training labels corrupted, and separately with 10% and 20% of the training images corrupted, comparing the final test accuracy against the clean baseline.

5. For the outlier study, take the first batch of 64 at initialization, compute s , multiply the first two entries by 100, and record the percentage change of each aggregator.
6. For the gradient study, differentiate each aggregator with respect to s itself in order to count the entries with $\partial L / \partial s_i \neq 0$, then differentiate with respect to θ to obtain the norm. Verify the median claim by comparing $\nabla_{\theta} L_{\text{MedSE}}$ against $\nabla_{\theta} s_{i_{\text{med}}}$ directly.
7. Repeat the clean training with batch sizes 16, 64 and 128 and compare the final accuracy and the mean gradient norm.

5 Implementation

The entire difference between the three experiments is contained in the two functions below.

```

1 def squared_errors(outputs, labels):
2     """Per sample squared error s_i = sum_c (y_ic - yhat_ic)^2 on
        one hot targets."""
3     targets = torch.zeros_like(outputs)
4     targets.scatter_(1, labels.unsqueeze(1), 1.0)
5     return ((targets - outputs) ** 2).sum(dim=1)
6
7
8 def aggregate(errors, kind):
9     if kind == "MSE":
10         return errors.mean()
11     if kind == "MedSE":
12         return errors.median()
13     if kind == "T-MSE":
14         count = errors.numel()
15         keep = count - int(TRIM_FRACTION * count)
16         ordered, _ = torch.sort(errors)
17         return ordered[:keep].mean()
18     raise ValueError(kind)

```

The count of samples that actually reach the parameters is obtained by backpropagating into a detached copy of the error vector, so that $\partial L / \partial s_i$ is materialised explicitly.

```

1 retained = errors.clone().detach().requires_grad_(True)
2 aggregate(retained, kind).backward()
3 weights = retained.grad
4 contributing = int((weights != 0).sum()).item()

```

6 Results

Loss	Train loss	Train acc. (%)	Test loss	Test acc. (%)	Epoch to 80%
MSE	0.2780	86.55	0.2858	85.70	3
MedSE	0.5742	52.52	0.5265	53.85	NR
T-MSE	0.2286	80.77	0.2165	80.70	5

Table 1: Experiment 1. Final epoch values on clean data with batch size 64. The last column is the first epoch at which test accuracy reaches 80 percent, and NR means the threshold was never reached within 5 epochs.

Corruption	Loss	Clean acc. (%)	Noisy acc. (%)	Degradation (pp)
Label noise 10%	MSE	85.70	85.70	+0.00
Label noise 10%	MedSE	53.85	60.55	+6.70
Label noise 10%	T-MSE	80.70	85.25	+4.55
Label noise 20%	MSE	85.70	84.75	-0.95
Label noise 20%	MedSE	53.85	62.45	+8.60
Label noise 20%	T-MSE	80.70	85.00	+4.30
Input noise 10%	MSE	85.70	85.70	+0.00
Input noise 10%	MedSE	53.85	64.35	+10.50
Input noise 10%	T-MSE	80.70	80.75	+0.05
Input noise 20%	MSE	85.70	85.80	+0.10
Input noise 20%	MedSE	53.85	53.90	+0.05
Input noise 20%	T-MSE	80.70	80.80	+0.10

Table 2: Experiments 2 and 3. Final test accuracy on the clean test set after training on corrupted data, with the degradation measured in percentage points against the clean baseline of the same loss.

Loss	Clean batch loss	With 2 outliers	Change (%)
MSE	2.573107	10.144366	+294.25
MedSE	2.573609	2.580293	+0.26
T-MSE	2.545176	2.556539	+0.45

Table 3: Experiment 4. Effect of multiplying the squared errors of 2 samples out of a batch of 64 by a factor of 100.

Loss	Batch loss	$\ \nabla_{\theta}L\ _2$	Samples with $\partial L/\partial s_i \neq 0$
MSE	2.573107	4.211414	64 / 64
MedSE	2.573609	8.045245	1 / 64
T-MSE	2.545176	4.109985	58 / 64

Table 4: Experiment 5. Gradient behaviour on one batch of 64 at initialization. The last column counts the entries of the batch with a nonzero $\partial L/\partial s_i$.

Batch	Loss	Final test acc. (%)	Mean $\ \nabla_{\theta}L\ _2$	Epoch to 80%
16	MSE	94.70	2.5150	1
16	MedSE	73.85	3.2334	NR
16	T-MSE	93.35	2.5477	1
64	MSE	85.70	1.5983	3
64	MedSE	53.85	5.3190	NR
64	T-MSE	80.70	1.6293	5
128	MSE	79.60	1.0003	NR
128	MedSE	53.10	5.0892	NR
128	T-MSE	75.95	1.0625	NR

Table 5: Experiment 6. Effect of batch size on clean data. The gradient norm is averaged over all optimizer steps of the run.

Quantity	MSE	MedSE	T-MSE
Clean test accuracy (%)	85.70	53.85	80.70
Label noise 10% degradation (pp)	+0.00	+6.70	+4.55
Label noise 20% degradation (pp)	-0.95	+8.60	+4.30
Input noise 10% degradation (pp)	+0.00	+10.50	+0.05
Input noise 20% degradation (pp)	+0.10	+0.05	+0.10
Epoch reaching 80% test accuracy	3	NR	5
Outlier sensitivity (% change)	+294.25	+0.26	+0.45
$\ \nabla_{\theta}L\ _2$ at initialization	4.21141	8.04524	4.10998
Samples driving the gradient (of 64)	64	1	58
Test accuracy at batch 16 (%)	94.70	73.85	93.35
Test accuracy at batch 64 (%)	85.70	53.85	80.70
Test accuracy at batch 128 (%)	79.60	53.10	75.95

Table 6: Summary of all six experiments.

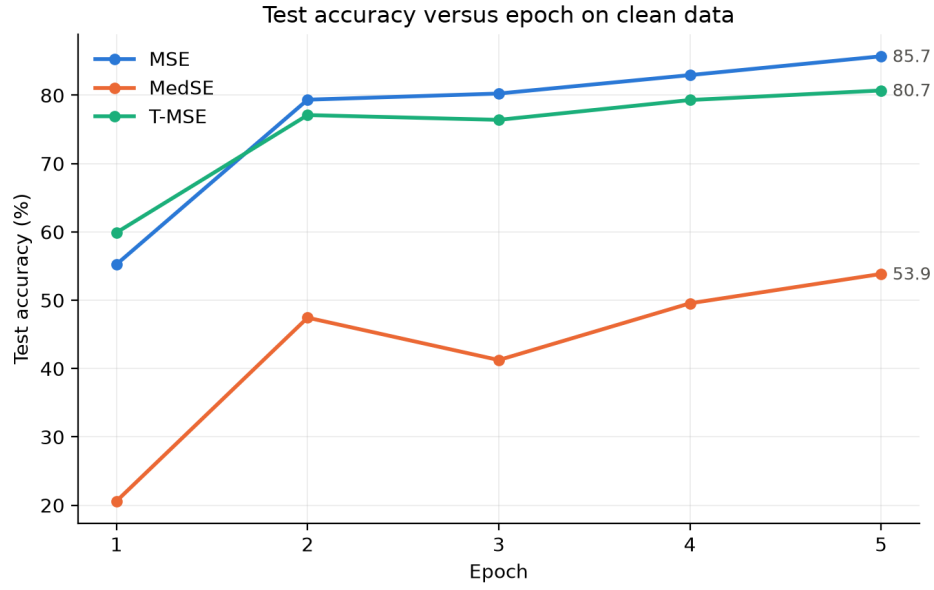


Figure 1: Test accuracy versus epoch on clean data, batch size 64.

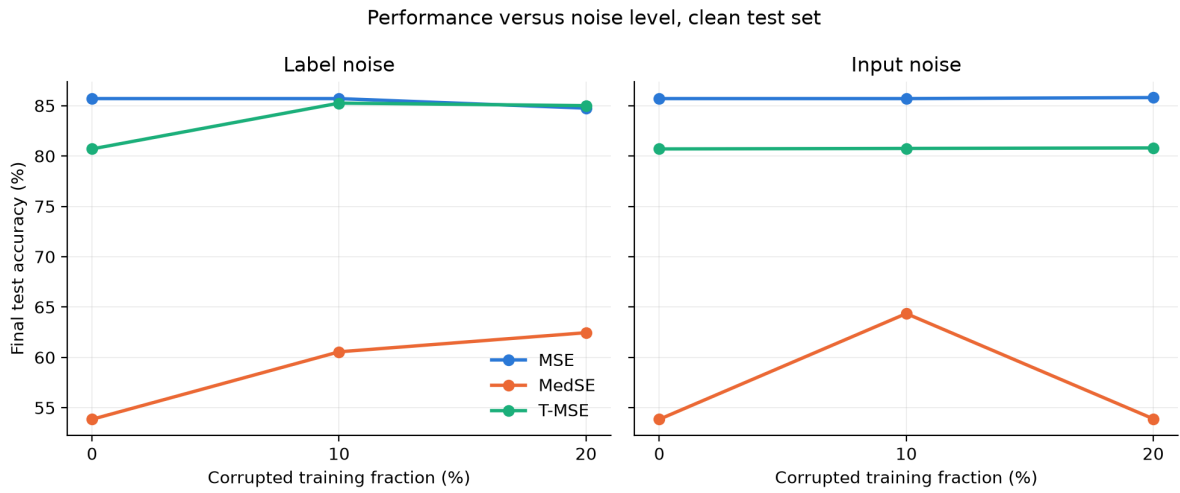


Figure 2: Final test accuracy on the clean test set versus the fraction of corrupted training data.

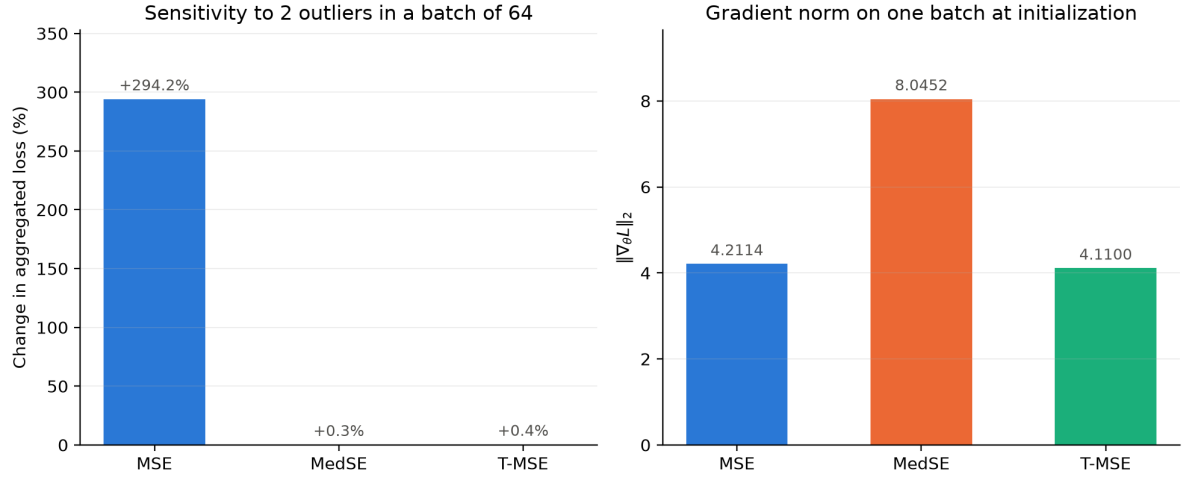


Figure 3: Left, the percentage change of each aggregated loss when 2 of 64 squared errors are multiplied by 100. Right, the parameter gradient norm produced by each aggregator on the same batch.

7 Analysis

7.1 Clean data costs the robust aggregators accuracy

On clean data the ordering is MSE at 85.70%, T-MSE at 80.70% and MedSE at 53.85%. MSE crosses 80% test accuracy at epoch 3 and T-MSE only at epoch 5, while MedSE never reaches it. This is the expected price of robustness under a clean distribution. The largest squared errors on clean MNIST belong to genuinely hard but correctly labelled digits, so discarding them discards signal. Trimming 10% costs about 5 percentage points at a fixed budget of 5 epochs, and the median, which effectively discards 63 of every 64 samples, costs about 32 points.

The MedSE curve in Figure 1 is also visibly non monotone, falling from 47.45% at epoch 2 to 41.25% at epoch 3 before recovering. Since each update is the gradient of a single sample, the update direction has the full variance of an individual example and the trajectory is correspondingly unstable.

7.2 Trimming pays for itself only when the tail is corrupted

The noise experiments invert the clean ordering. Under 20% label noise MSE falls by 0.95 points to 84.75% while T-MSE rises by 4.30 points to 85.00% and therefore overtakes MSE. The mechanism is exactly the one predicted by the breakdown argument: a mislabeled sample produces a large squared error because the network’s sigmoid output agrees with the true class and disagrees with the corrupted target, so mislabeled samples concentrate in the upper tail of s and are precisely what the trim removes. At 20% corruption the trim is only 10% wide, so it removes roughly half of the mislabeled samples, which is enough to recover the gap. The same effect is visible in the training accuracy on the corrupted set, where MSE reaches 68.17% against T-MSE at 68.63% while both generalize to around 85%, that is, neither has begun to memorize the wrong labels within 5 epochs.

Input noise of standard deviation 0.1 leaves MSE and T-MSE essentially unchanged, within 0.1 points at both levels. A perturbation of this size on normalized MNIST behaves as mild data augmentation rather than as an outlier source, and it does not move the affected samples into the tail of the error distribution, so there is nothing for a robust aggregator to reject. This is a useful negative control: robust aggregation helps against corrupted targets, not against mildly corrupted inputs.

The MedSE column of the noise table should be read with care. Its apparent gains of 6.70 to 10.50 points are measured from a baseline that is far from converged, and the input noise response is non monotone, rising to 64.35% at 10% corruption and returning to 53.90% at 20%. These are run to run fluctuations of an unstable optimizer rather than evidence of robustness.

7.3 Outlier sensitivity follows the breakdown point exactly

Two inflated samples out of 64 raise MSE by 294.25%, which matches Equation 5 once the actual share of those two samples in the batch error, about 3.0%, is substituted. The median moves by 0.26% and the trimmed mean by 0.45%. Neither moves by the magnitude of the corruption, only by the rank displacement it causes. For the trimmed mean the residual 0.45% has a precise cause: the two inflated samples were not originally among the six largest, so inflating them pushes them into the trimmed set and pushes two

previously trimmed samples back into the retained average, which changes the retained mean slightly. Since $2/64 = 3.1\%$ is well below the 10% breakdown point, the corruption itself is fully rejected.

7.4 The median does produce gradients, but from one sample

`backward()` succeeds for MedSE. PyTorch treats `median` as a selection and routes the entire incoming gradient to the selected index, so the count of entries with $\partial L / \partial s_i \neq 0$ is 1 out of 64, against 58 out of 64 for T-MSE and 64 out of 64 for MSE. The claim was verified directly rather than inferred: the median sample of the batch was sample index 11 of the batch, with $\|\nabla_{\theta} L_{\text{MedSE}} - \nabla_{\theta} s_{11}\|_2 = 0$, that is, the two gradients are bitwise identical. The gradient is therefore affected by only the median loss sample and not by the complete batch.

The norms confirm the variance argument of Section 2.3. MedSE produces $\|\nabla_{\theta} L\|_2 = 8.045$ against 4.211 for MSE on the same batch and from the same weights, because MSE averages 64 partially cancelling per sample gradients while MedSE returns one of them undiluted. A larger gradient norm here signals higher variance, not faster learning, which is consistent with MedSE being both the noisiest and the slowest of the three.

7.5 Batch size separates the aggregators by an order of magnitude

At a fixed budget of 5 epochs, increasing the batch size reduces the number of optimizer steps from 3125 to 785 to 395, and all three losses degrade: MSE falls from 94.70% to 85.70% to 79.60%. The median falls much further in relative terms, from 73.85% to 53.85% to 53.10%, and this is the predicted effect of Section 2.4. A median run applies gradient information from one sample per step, so across the whole run it uses 3125, 785 and 395 sample presentations respectively, against 50000 for MSE at every batch size. Its effective data efficiency is $1/n$.

The averaged gradient norms make the same point from the other side. For MSE the mean norm falls from 2.515 to 1.598 to 1.000 as the batch grows, close to the $1/\sqrt{n}$ behaviour expected from averaging uncorrelated per sample gradients. For MedSE it does not fall at all, standing at 3.233, 5.319 and 5.089, because no averaging takes place. Larger batches therefore buy the mean a better conditioned gradient and buy the median nothing except a proportionally smaller fraction of the data.

8 Research Hypothesis

Two findings sit next to each other in the tables and suggest the same remedy. Trimming helps exactly when the discarded tail is corrupted, costing 5.0 points on clean data and gaining 4.3 points at 20% label noise, and the median throws away the batch average in order to buy a breakdown point it does not need at 3% contamination.

Hypothesis. *A trimmed aggregator whose trim fraction is matched to the actual contamination rate, and whose trimming is switched on only after the per sample error distribution separates into two modes, will dominate both MSE and a fixed 10% trimmed mean at every noise level, matching MSE on clean data and exceeding it under label noise.*

The reasoning is that early in training all squared errors are large and a clean hard sample is statistically indistinguishable from a mislabeled one, so an early trim removes informative examples at random; only once the network has fitted the clean majority do the mislabeled samples separate into a distinct upper mode. The contamination rate can be estimated online from the bimodality of the batch error histogram, which makes the scheme self tuning rather than another hyperparameter.

A second and independent hypothesis follows from the gradient results. *Replacing the median by the mean of the central half of the batch, that is the interquartile mean, will retain the median’s rejection of inflated errors while restoring the $1/\sqrt{n}$ gradient variance of the mean, and will therefore close most of the 32 point accuracy gap between MedSE and MSE without reintroducing MSE’s zero breakdown point.* This is testable within the present setup by changing one line of **aggregate**, and the predicted signature is a mean gradient norm that falls with batch size in the way MSE’s does rather than staying flat in the way MedSE’s does.

9 Conclusion

Under one fixed protocol the three aggregators separate cleanly along the robustness and optimization trade off. The mean is the fastest and most accurate on clean data at 85.70% but has breakdown point zero, and two inflated samples in a batch of 64 move its value by 294%. The 10% trimmed mean gives up 5.0 points on clean data in exchange for a 10% breakdown point, rejects the same outliers to within 0.45%, uses 58 of every 64 samples for its gradient, and overtakes the mean once 20% of the labels are wrong. The median is robust for the wrong reason: its breakdown point of 50% is far more than the problem requires, and it is paid for by using one sample per update, which leaves it at 53.85%, non monotone in the epoch curve, and degrading in direct proportion to batch size. For the corruption levels studied here the trimmed mean is the only aggregator that is robust and trainable at the same time, and the practical conclusion is that the trim fraction, not the choice of aggregator family, is the quantity that should be tuned to the data.